

RoyalRent

Plataforma de Alquiler de Vehículos y Servicios Premium

Proyecto Fin de Ciclo

Desarrollo de Aplicaciones Multiplataforma (DAM)

Autores:

Gonzalo Velasco & Miguel José

Curso: 2024-2025

IES · Ciclo Formativo de Grado Superior en DAM

1. Índice

1. Introducción

- Stack Tecnológico
- Estructura del Proyecto

2. Frontend

- main.jsx
- App.jsx
- Navbar.jsx
- Footer.jsx
- Home.jsx
- Services.jsx
- Auth.jsx
- Cart.jsx
- Profile.jsx
- AuthContext / useAuth
- CartContext / useCart
- index.css y servicios.css

3. Backend

- server.js
- schema.sql
- init_db.js

4. Base de Datos

5. API Endpoints

- 6. Diagramas de Flujo
- 7. Cómo Ejecutar el Proyecto
- 8. Decisiones Técnicas

1. Introducción

RoyalRent es una aplicación web desarrollada como Proyecto Fin de Ciclo del Grado Superior en Desarrollo de Aplicaciones Multiplataforma (DAM). Consiste en una plataforma de alquiler de vehículos y servicios premium que permite a los usuarios explorar un catálogo de coches de lujo, yates y helicópteros, gestionar un carrito de compra, registrarse, iniciar sesión y consultar su historial de reservas.

El proyecto se divide en dos partes claramente diferenciadas:

- **Frontend:** Aplicación web SPA (Single Page Application) construida con React 19 y Vite 8.
- **Backend:** API REST construida con Node.js, Express 5 y MySQL, que proporciona los endpoints necesarios para la gestión de usuarios, productos y pedidos.

Objetivo académico: Demostrar la capacidad de diseñar, desarrollar y documentar una aplicación completa full-stack, aplicando los conocimientos adquiridos durante el ciclo formativo en programación, bases de datos, diseño de interfaces y despliegue.

2. Stack Tecnológico

Frontend

Tecnología	Versión	Función
React	19.2.5	Biblioteca principal para la construcción de la interfaz de usuario basada en componentes
Vite	8.0.10	Entorno de desarrollo y empaquetador (bundler) ultrarrápido
React Router	7.15.0	Enrutamiento del lado del cliente (navegación SPA)
Axios	1.16.0	Cliente HTTP para las peticiones al backend

Lucide React	1.14.0	Iconos vectoriales ligeros para la interfaz
Font Awesome	6.5.0	Iconos adicionales (CDN)
ESLint	10.2.1	Análisis estático de código para mantener calidad y buenas prácticas

Backend

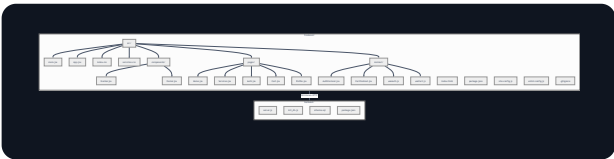
Tecnología	Versión	Función
Node.js	—	Entorno de ejecución JavaScript del lado del servidor
Express	5.2.1	Framework web para la creación de la API REST
MySQL2	3.22.3	Driver de conexión a la base de datos MySQL con soporte de promesas
JSON Web Token (JWT)	9.0.3	Autenticación mediante tokens cifrados
bcrypt	6.0.0	Hash y comparación segura de contraseñas
CORS	2.8.6	Middleware para permitir peticiones desde el frontend (cross-origin)
dotenv	17.4.2	Carga de variables de entorno desde archivo .env

Base de Datos

Tecnología	Función
MySQL	Base de datos relacional que almacena usuarios, productos y pedidos
XAMPP / MySQL Workbench	Entorno de gestión local de la base de datos

3. Estructura del Proyecto

El proyecto se organiza en dos directorios independientes: `frontend/` y `backend/`. Cada uno tiene su propio `package.json` y sus dependencias.



Descripción de Directorios y Archivos

Ruta	Descripción
frontend/index.html	Punto de entrada HTML. Carga el módulo principal de React y las fuentes/iconos externos
frontend/src/main.jsx	Punto de entrada de React. Renderiza el componente App dentro de StrictMode
frontend/src/App.jsx	Componente raíz. Define el Router, los providers de contexto y las rutas
frontend/src/components/Navbar.jsx	Barra de navegación superior con menú responsive, login/logout y acceso al carrito
frontend/src/components/Footer.jsx	Pie de página con enlaces a servicios, equipo y contacto
frontend/src/pages/Home.jsx	Página principal con hero, reseñas, servicios destacados, equipo y formulario de contacto
frontend/src/pages/Services.jsx	Catálogo de servicios con filtros por categoría y tarjetas de producto
frontend/src/pages/Auth.jsx	Página de inicio de sesión y registro con formulario único conmutante
frontend/src/pages/Cart.jsx	Carrito de compra con dos pasos: revisión de items y pantalla de pago simulado
frontend/src/pages/Profile.jsx	Perfil de usuario con formulario de actualización de datos e historial de reservas
frontend/src/context/AuthContext.jsx	Contexto de autenticación: login, registro, logout, persistencia del token y carga del perfil
frontend/src/context/CartContext.jsx	Contexto del carrito: añadir, eliminar, vaciar y calcular total
frontend/src/context/useAuth.js	Hook para acceder al AuthContext
frontend/src/context/useCart.js	Hook para acceder al CartContext

<code>frontend/src/index.css</code>	Estilos globales: variables CSS, navbar, hero, secciones, footer, responsive
<code>frontend/src/servicios.css</code>	Estilos específicos del catálogo: filtros, tarjetas, grid de productos
<code>backend/server.js</code>	Servidor Express con todas las rutas de la API (auth, usuarios, items, pedidos)
<code>backend/schema.sql</code>	Script SQL de creación de la base de datos y tablas, con datos de prueba
<code>backend/init_db.js</code>	Script Node.js que ejecuta schema.sql automáticamente

4. Frontend

main.jsx

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.jsx'
import './index.css'
import './servicios.css'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)
```

Fichero de entrada de la aplicación. **React.StrictMode** activa comprobaciones en desarrollo (doble renderizado, detección de efectos incorrectos). Se importan los dos archivos CSS globales (`index.css` y `servicios.css`) para que estén disponibles en toda la app.

App.jsx

```
<AuthProvider>
  <CartProvider>
    <Router>
      <Navbar />
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/services" element={<Services />} />
        <Route path="/auth" element={<Auth />} />
        <Route path="/profile" element={<Profile />} />
        <Route path="/cart" element={<Cart />} />
      </Routes>
      <Footer />
    </Router>
  </CartProvider>
</AuthProvider>
```

Componente raíz que define la estructura de la aplicación. Los providers (`AuthProvider` y `CartProvider`) envuelven toda la app para que cualquier componente hijo pueda acceder al estado global de autenticación y carrito mediante hooks.

Se utiliza `BrowserRouter` de React Router v7 para la navegación SPA con 5 rutas. El `Navbar` y `Footer` se renderizan siempre, mientras que el contenido cambia según la ruta.

Navbar.jsx

Barra de navegación superior con las siguientes funcionalidades:

- **Scroll detection:** Un `useEffect` escucha el evento `scroll` del window. Cuando el scroll supera 50px, añade la clase `solid` para cambiar el fondo de la navbar (de transparente a sólido). Se limpia el listener al desmontar.
- **Menú responsive:** Un botón hamburguesa (`burger`) togglea el menú en móviles mediante `menuOpen` state.
- **Autenticación:** Muestra el nombre del usuario y botón de logout si está logueado, o un enlace "Login" si no.
- **Carrito:** Icono con badge que muestra el número de items en el carrito.
- **Logo dual:** Muestra un logo para desktop y otro (favicon) para móviles mediante CSS.

🚀 Detalle técnico: smooth scroll ocultación

La navbar arranca con la clase `nav-hidden-top` que la oculta desplazándola hacia arriba con `transform: translateY(-100%)`. Al hacer scroll, la clase `solid` la muestra. Esto crea el efecto de "revelar la navbar al hacer scroll".

Footer.jsx

Pie de página simple con el logo, copyright (Gonzalo Velasco & Miguel José · PFC DAM) y enlaces rápidos a Servicios, Equipo y Contacto.

Home.jsx

Página principal con 5 secciones:

1. **Hero:** Video de fondo (con poster de respaldo), título principal "El Lujo que Mereces", botón CTA a servicios, estadísticas (500+ servicios, 3 categorías, 24/7 soporte) y un indicador de scroll animado.
2. **Confianza (Reviews):** Un marquee horizontal con reseñas simuladas de Google. Cada card contiene avatar, nombre, fecha, valoración (estrellas) y texto de opinión. El track se desplaza infinitamente con animación CSS y se pausa al hacer hover.
3. **Servicios:** Grid de 3 tarjetas que enlazan a `/services?type=car|yacht|helicopter` pasando la categoría como query param.
4. **Equipo:** Dos tarjetas presentando a los desarrolladores con sus roles y tecnologías usadas (tags).
5. **Contacto:** Formulario de contacto (nombre, email, mensaje) visual, sin lógica de envío implementada (es meramente estructural).

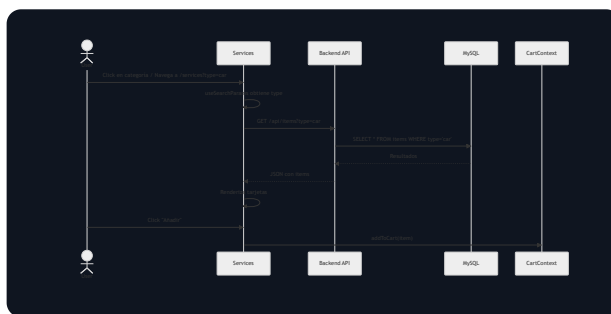
Nota: El formulario de contacto no tiene manejo de submit porque es una maqueta visual. En una versión real se conectaría a un servicio de email o se almacenaría en BD.

Services.jsx

Catálogo de servicios con las siguientes características:

- **Filtrado por URL:** Utiliza `useSearchParams` de React Router para leer el query param `type` (`?type=car`, `?type=yacht`, `?type=helicopter`). Los enlaces desde Home.jsx ya incluyen este parámetro.

- **Pestañas de filtro:** 4 botones (Todos, Coches, Yates, Helicópteros) que actualizan el query param con `setSearchParams`.
- **Fetch de datos:** Un `useEffect` dependiente del filtro realiza una petición GET a `/api/items` con o sin `?type=` según corresponda. Los resultados se almacenan en el state `items`.
- **Tarjetas de producto:** Cada item se renderiza como una `listing-card` con imagen de fondo (con fallback si no hay URL), nombre, descripción, estrellas (estáticas), especificaciones según el tipo, precio y botón "Añadir" que llama a `addToCart` del contexto.



Auth.jsx

Página de autenticación que combina login y registro en un mismo componente:

- **Toggle login/registro:** Un state `isLogin` alterna entre los dos modos. En modo registro se muestra un campo adicional "Nombre completo".
- **Formulario controlado:** `formData` (name, email, password) se actualiza con `onChange` en cada input.
- **Manejo de errores:** Si el servidor devuelve error (email duplicado, credenciales inválidas), se muestra un mensaje en un banner rojo.
- **Redirección:** Tras login/registro exitoso, navega a `/profile`.
- **Estado loading:** Mientras se procesa la petición, el botón se deshabilita y muestra "Procesando..."

Cart.jsx

Carrito de compra con un flujo de 2 pasos:

1. **Paso 1 — Carrito:** Lista los items añadidos con imagen, nombre, cantidad y precio. Muestra el total. Botón "Proceder al Pago Seguro" que cambia al paso 2 (si no hay usuario logueado, redirige a login).
2. **Paso 2 — Pago:** Pantalla simulada de pago con selección de tarjeta (Visa, Mastercard, o añadir nueva). Un resumen lateral muestra subtotal, tasas y total. El botón "Pagar" ejecuta

```
handleCheckout .
```

El **checkout** envía un POST a `/api/orders` con los datos del carrito. Si tiene éxito, limpia el carrito y redirige al perfil con una alerta de confirmación.

Profile.jsx

Perfil de usuario con dos paneles:

- **Izquierda — Actualizar datos:** Formulario precargado con nombre, email y contraseña opcional. Al enviar, hace PUT a `/api/user/profile` y muestra mensaje de éxito/error.
- **Derecha — Historial de reservas:** Fetch a `/api/orders/history`. Cada orden se muestra como una card con referencia (#XXXXX), total pagado y lista de items (imagen, nombre, tipo, cantidad, precio). Si no hay reservas, muestra un mensaje con enlace al catálogo.
- **Logout:** Botón que llama a `logout()` del contexto y redirige al home.
- **Protección:** Si no hay usuario (`!user`), redirige a `/auth`. Mientras tanto, retorna `null`.

AuthContext.jsx / useAuth.js

Contexto de autenticación que provee el estado del usuario a toda la aplicación:

🔴 Flujo de autenticación

1. Al cargar la app, se comprueba si hay un **token** en `localStorage`.
2. Si existe, se añade a los headers de Axios y se hace `GET /api/user/profile` para obtener los datos del usuario.
3. Si la petición falla, se ejecuta `logout()` (token inválido/expirado).
4. Si no hay token, `loading` se inicializa a `false` y se renderiza la app sin usuario.
5. Las funciones `login` y `register` hacen peticiones al backend, almacenan el token y actualizan el estado.

Funciones expuestas:

Función	Descripción
<code>login(email, password)</code>	POST <code>/api/login</code> , guarda token en <code>localStorage</code> , actualiza estado
<code>register(name, email, password)</code>	POST <code>/api/register</code> , luego hace login automático
<code>logout()</code>	Limpia token de <code>localStorage</code> , resetea estado, elimina header de Axios

`setUser(user)`

Actualiza el usuario manualmente (usado en Profile al editar datos)

Nota: El hook `useAuth` se ha separado en un archivo independiente (`useAuth.js`) para evitar errores de Fast Refresh de ESLint, ya que React requiere que los archivos exporten solo componentes para un HMR óptimo.

CartContext.jsx / useCart.js

Contexto del carrito que mantiene el estado de los items seleccionados:

Función	Descripción
<code>addToCart(item)</code>	Añade un item al carrito. Si ya existe, incrementa su cantidad en 1
<code>removeFromCart(id)</code>	Elimina completamente un item del carrito por su id
<code>clearCart()</code>	Vacía el carrito por completo
<code>cartTotal</code>	Valor calculado: suma de (precio × cantidad) de todos los items

index.css y servicios.css

Dos archivos CSS que siguen una arquitectura modular:

- **index.css:** Variables globales (paleta de colores, fuentes, sombras, bordes), reset, estilos de la navbar, hero, secciones, botones, grid de servicios, reseñas, equipo, formularios, footer y media queries responsive hasta 640px.
- **servicios.css:** Estilos específicos del catálogo: breadcrumb, filtros (tabs), grid de tarjetas de producto, especificaciones, precios, botón de añadir al carrito y responsive.

Variables CSS principales:

```
:root {
  --accent: #c5a059;           /* Dorado corporativo */
  --surface: #ffffff;          /* Fondos claros */
  --bg: #f5f6f8;              /* Fondo general */
  --text: #0f1520;            /* Texto principal */
  --text-2: #4a5568;          /* Texto secundario */
  --serif: 'Playfair Display'; /* Fuente para títulos */
  --sans: 'Inter';             /* Fuente para texto */
}
```

5. Backend

server.js

Servidor Express que expone una API REST en el puerto 5000. Estructura general:

```
// Configuración inicial
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const mysql = require('mysql2/promise');

const app = express();
app.use(cors());
app.use(express.json());

// Pool de conexiones MySQL
const pool = mysql.createPool({
  host: process.env.DB_HOST || 'localhost',
  user: process.env.DB_USER || 'root',
  password: process.env.DB_PASSWORD || '',
  database: process.env.DB_NAME || 'royalrent',
  waitForConnections: true,
  connectionLimit: 10
});
```

Se utiliza un **pool de conexiones** (límite de 10 conexiones simultáneas) para no abrir/cerrar una conexión en cada petición, lo que mejora el rendimiento.

Middleware de autenticación: authenticateToken

```
const authenticateToken = (req, res, next) => {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];
  if (!token) return res.sendStatus(401);

  jwt.verify(token, process.env.JWT_SECRET || 'secret_key', (err, user) => {
    if (err) return res.sendStatus(403);
    req.user = user;
    next();
  });
};
```

Extrae el token del header `Authorization: Bearer <token>`, lo verifica con `jwt.verify` y, si es válido, adjunta los datos del usuario al objeto `req`. Si no hay token → 401. Si el token es inválido/expirado → 403.

Rutas de autenticación

Ruta	Método	Autenticación	Lógica
<code>/api/register</code>	POST	No	Recibe name, email, password. Hashea la contraseña con bcrypt (10 rondas). Inserta en BD. Si el email ya existe, devuelve 400 (ER_DUP_ENTRY).
<code>/api/login</code>	POST	No	Busca el usuario por email. Compara contraseña con bcrypt. Si coincide, genera un JWT con expiración de 24h y lo devuelve junto con los datos del usuario.

Rutas de usuario

Ruta	Método	Autenticación	Lógica
<code>/api/user/profile</code>	GET	Sí	Devuelve id, name, email, created_at del usuario autenticado.
<code>/api/user/profile</code>	PUT	Sí	Actualiza name, email y opcionalmente password del usuario autenticado. Si se envía password, lo hashea antes de actualizar.

Rutas de items

Ruta	Método	Autenticación	Lógica
<code>/api/items</code>	GET	No	Devuelve todos los items. Si se pasa query param <code>?type=car yacht helicopter</code> , filtra por tipo.

Rutas de pedidos

Ruta	Método	Autenticación	Lógica
<code>/api/orders</code>	POST	Sí	Crea un pedido en una transacción: inserta en <code>orders</code> (user_id, total_price) y en <code>order_items</code> (order_id, item_id, quantity, price) para cada item. Si algo falla, hace rollback.
<code>/api/orders/history</code>	GET	Sí	Devuelve todos los pedidos del usuario autenticado ordenados por fecha descendente. Cada pedido incluye sus items con nombre, tipo e imagen mediante un JOIN con la tabla items.

Detalle técnico: Transacciones en checkout

La ruta POST `/api/orders` utiliza una **transacción SQL** (`BEGIN TRANSACTION` / `COMMIT` / `ROLLBACK`) para garantizar que la orden y sus items se inserten de forma atómica. Si falla cualquier inserción, se deshacen todos los cambios y se devuelve error 500.

schema.sql

Script SQL que define la estructura completa de la base de datos:

```
CREATE TABLE IF NOT EXISTS users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  password VARCHAR(255) NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE IF NOT EXISTS items (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  type ENUM('car', 'yacht', 'helicopter') NOT NULL,  
  name VARCHAR(100) NOT NULL,  
  description TEXT,  
  price DECIMAL(10,2) NOT NULL,  
  image_url VARCHAR(255),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE IF NOT EXISTS orders (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  user_id INT NOT NULL,  
  total_price DECIMAL(10,2) NOT NULL,  
  status VARCHAR(50) DEFAULT 'completed',  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (user_id) REFERENCES users(id)  
);  
  
CREATE TABLE IF NOT EXISTS order_items (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  order_id INT NOT NULL,  
  item_id INT NOT NULL,  
  quantity INT DEFAULT 1,  
  price DECIMAL(10,2) NOT NULL,  
  FOREIGN KEY (order_id) REFERENCES orders(id),  
  FOREIGN KEY (item_id) REFERENCES items(id)  
);
```

Además de las tablas, el script inserta **40 registros de prueba**: 20 coches, 12 yates y 8 helicópteros, con nombres reales de fabricantes (Ferrari, Lamborghini, Rolls-Royce, Azimut, Sunseeker, Bell, Airbus, etc.), descripciones, precios e imágenes.

init_db.js

Script que automatiza la creación de la base de datos. Su funcionamiento:

1. Se conecta a MySQL **sin especificar base de datos** (para poder crearla).

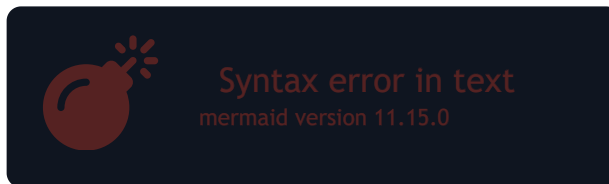
2. Ejecuta `DROP DATABASE IF EXISTS royalrent` (borra la anterior si existe).
3. Lee el archivo `schema.sql` del disco.
4. Divide el contenido por punto y coma y ejecuta cada sentencia.
5. Muestra mensajes de estado en consola.

⚠ Advertencia

`init_db.js` **borra y recrea** la base de datos desde cero. Al ejecutarlo se pierden todos los datos existentes. Está pensado para la inicialización inicial o para resetear la BD durante desarrollo.

6. Base de Datos

Diagrama Entidad-Relación



Descripción de Tablas

Tabla	Descripción	Campos clave
<code>users</code>	Almacena los usuarios registrados	<code>id</code> (PK, autoincremental), <code>email</code> (UNIQUE), <code>password</code> (hasheado con bcrypt)
<code>items</code>	Catálogo de productos disponibles para alquilar	<code>type</code> (ENUM: car/yacht/helicopter), <code>price</code> (DECIMAL con 2 decimales), <code>image_url</code>
<code>orders</code>	Pedidos realizados por los usuarios	<code>user_id</code> (FK → users), <code>total_price</code> , <code>status</code> (default: 'completed')
<code>order_items</code>	Items individuales dentro de cada pedido (relación N:M entre orders e items)	<code>order_id</code> (FK → orders), <code>item_id</code> (FK → items), <code>quantity</code> , <code>price</code>

📌 Relaciones

- **users** → **orders**: 1:N. Un usuario puede tener múltiples pedidos.
- **orders** → **order_items**: 1:N. Un pedido contiene múltiples items.
- **items** → **order_items**: 1:N. Un producto puede aparecer en múltiples pedidos.

7. API Endpoints

Resumen completo de todas las rutas disponibles en la API REST:

Método	Ruta	Auth	Body / Query	Respuesta	Códigos
POST	/api/register	No	{ name, email, password }	{ message, userId }	201 / 400 / 500
POST	/api/login	No	{ email, password }	{ token, user }	200 / 401 / 500
GET	/api/user/profile	Sí	—	{ id, name, email, created_at }	200 / 401 / 403 / 404
PUT	/api/user/profile	Sí	{ name, email, password? }	{ message }	200 / 500
GET	/api/items	No	? type=car yacht helicopter (opcional)	[{ id, type, name, description, price, image_url }]	200 / 500
POST	/api/orders	Sí	{ items: [{ id, quantity, price }], totalPrice }	{ message, orderId }	201 / 500

GET

/api/orders/history

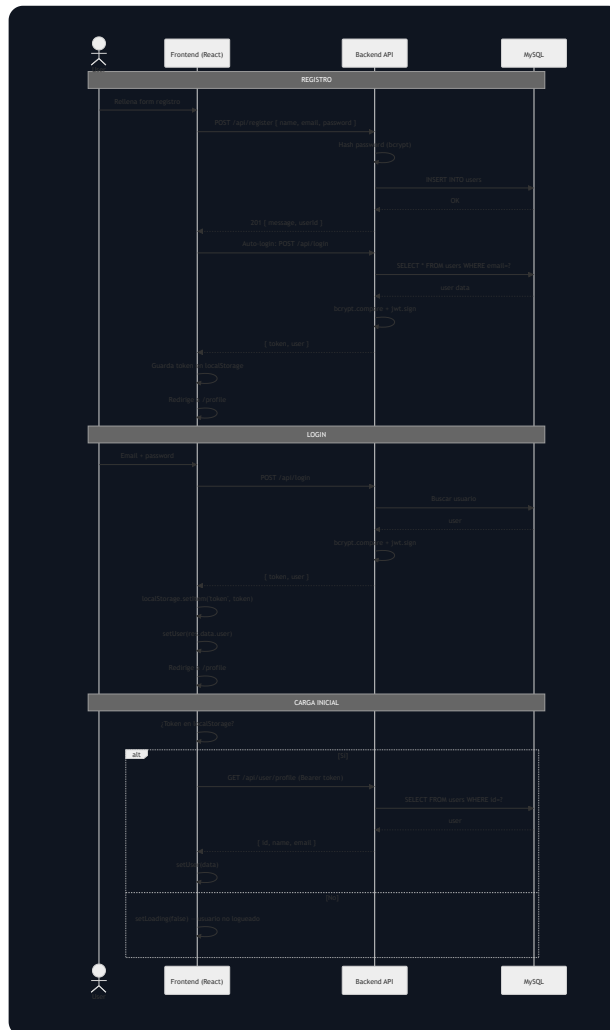
Sí

—

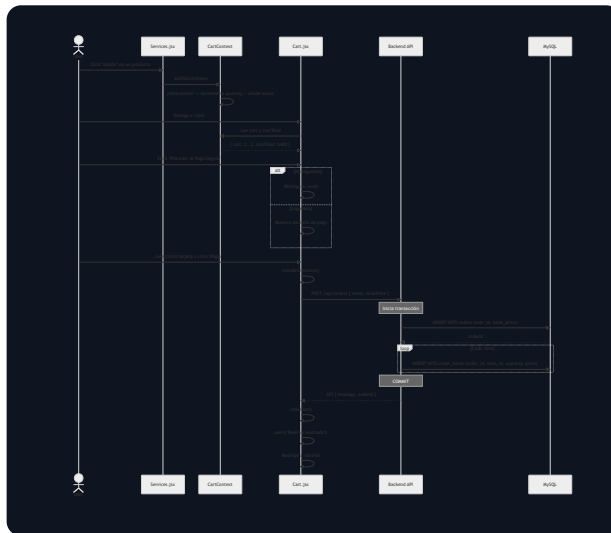
```
[{ id,
  total_price,
  status,
  created_at,
  items: [...]}]
```

8. Diagramas de Flujo

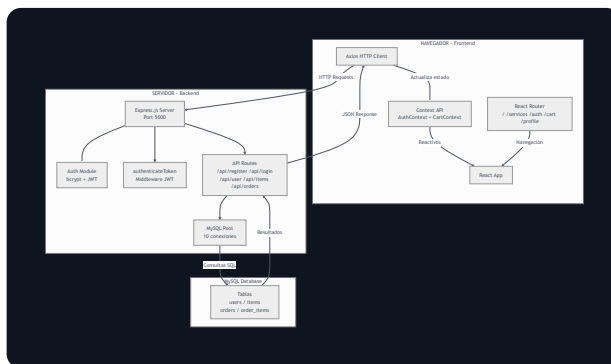
Flujo de Autenticación



Flujo del Carrito y Checkout



Arquitectura Frontend ↔ Backend



9. Cómo Ejecutar el Proyecto

Prerrequisitos

- **Node.js** (v18 o superior)
- **MySQL** (XAMPP, MySQL Workbench o similar)
- **npm** (incluido con Node.js)

Paso a paso

1. Iniciar MySQL

Asegúrate de que el servicio de MySQL esté ejecutándose (XAMPP → Start MySQL).

2. Configurar variables de entorno (opcional)

Crea un archivo `.env` en la carpeta `backend/` :

```
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=
DB_NAME=royalrent
JWT_SECRET=tu_clave_secreta
PORT=5000
```

Si no se crea, el servidor usa los valores por defecto.

3. Inicializar la base de datos

```
cd backend
npm install
npm run init-db
```

Esto ejecuta `init_db.js` , que crea la BD, las tablas e inserta los 40 productos de prueba.

4. Iniciar el backend

```
cd backend
npm start
```

El servidor arranca en `http://localhost:5000` .

5. Iniciar el frontend

```
cd frontend
npm install
npm run dev
```

La aplicación se abre en `http://localhost:5173` .

10. Decisiones Técnicas

¿Por qué React y no otro framework?

React ofrece un modelo basado en componentes que facilita la reutilización de código, tiene una comunidad muy activa y su ecosistema (React Router, Axios) es maduro. Para un proyecto de este tamaño, React proporciona el equilibrio perfecto entre potencia y simplicidad. Se eligió Vite como bundler por su rapidez frente a Create React App (Webpack).

¿Por qué Express y no otros como Fastify o NestJS?

Express es el framework web más extendido para Node.js, con la documentación más amplia y el mayor número de tutoriales. Para una API REST de este tamaño, Express ofrece exactamente lo necesario sin sobrecarga de abstracciones adicionales.

¿Por qué MySQL y no MongoDB?

Los datos del proyecto (usuarios, productos, pedidos) tienen relaciones claras: un pedido tiene items, un item pertenece a una categoría. Una base de datos relacional como MySQL es la elección natural para modelar estas relaciones mediante claves foráneas y transacciones ACID. Además, MySQL es el estándar en el ciclo formativo DAM.

¿Por qué JWT para autenticación?

JWT permite autenticación sin estado (stateless): el servidor no necesita almacenar sesiones, el token contiene toda la información necesaria. Esto es ideal para APIs REST y facilita la escalabilidad. El token se guarda en `localStorage` del navegador y se envía en cada petición mediante el header `Authorization`.

¿Por qué bcrypt para contraseñas?

bcrypt aplica un hash con sal (salt) incorporada y es deliberadamente lento, lo que dificulta los ataques de fuerza bruta. Con 10 rondas de hasheo, se garantiza que incluso si la base de datos se ve comprometida, las contraseñas no pueden ser recuperadas.

¿Por qué contexto (Context API) en lugar de Redux?

El proyecto solo necesita dos estados globales simples (usuario y carrito). La Context API de React es suficiente para este caso de uso sin añadir la complejidad de Redux (store, reducers, actions,

middlewares).

¿Por qué separar frontend y backend?

La separación en dos proyectos independientes (frontend/ y backend/) sigue el patrón de arquitectura de microservicios a pequeña escala. Esto permite: desarrollar y desplegar cada parte por separado, reutilizar el backend con otros frontends (por ejemplo, una app Android), y mantener un código más organizado.

¿Por qué transacciones SQL en el checkout?

Al crear un pedido, se insertan datos en dos tablas (orders y order_items). Si falla la inserción de un item después de haber insertado la orden, la base de datos quedaría inconsistente. Las transacciones garantizan que todas las operaciones se completen o ninguna.

¿Por qué pool de conexiones MySQL?

Abrir y cerrar una conexión MySQL en cada petición es costoso. Un pool mantiene un conjunto de conexiones reutilizables, mejorando el rendimiento y evitando saturar la base de datos.